

kind of regular expression syntax being used in C++. Sometimes it is better to attack the problem directly than beat around the bush. But even then, it is absolutely essential to know the regular expression syntax used with C++. Otherwise, this article may just be a set of 'Do-It-Yourself' instructions. C++11 regular expressions support multiple regular expression styles like ECMAScript syntax, AWK script syntax, Grep syntax, etc.

ECMAScript is a scripting language and JavaScript is the most well-known implementation of ECMAScript. The syntax used by ECMAScript is not much different from the other regular expression flavours. There are some minor differences, though. For example, the notation `ld` used in Perl style regular expressions to denote decimal digits is absent in ECMAScript style regular expressions. Instead, a notation like `[[:digit:]]` is used there. I am not going to point out any other such difference but just keep in mind that C++11 supports multiple regular expression styles and some of the styles differ from the others, slightly.

A practical regular expression for C++

Now let us discuss a practical regular expression with which we can find out some real data rather than finding out 'strings starting with *abc* and ending with *xyz*'. Our aim is to identify those lines that contain only numbers. Consider the text file *file2.txt* with the following data to test our regular expressions:

```
abcxyz
a1234z
111222
123456
aaaaaaa
zzzzzzz
AA111
111
2222
33333
22.22
8888
```

Now consider the program *regex7.cc* with the following code:

```
#include <iostream>
#include <string>
#include <fstream>
#include <regex>

using namespace std;

int main()
{
    ifstream file("file2.txt");
    string str;
    regex pat("^[[:digit:]]+$");
```

```
while (getline(file, str))
{
    if ( regex_search(str,pat) )
    {
        cout << str << "\n";
    }
    return 0;
}
```

On execution with the following commands, *g++ regex7.cc* and *./a.out* the program prints those lines containing numbers alone. Figure 4 shows the output of the program *regex7.cc*.

Except for the line of code *regex pat("^[[:digit:]]+\$");* which defines the regular expression pattern to be searched, there is no difference between the working of the programs *regex5.cc* and *regex7.cc*. The caret symbol `^` is used to denote that the match should happen at the very beginning and the dollar symbol `$` is used to denote that the match should occur at the end. In the middle, there is the regular expression `[[:digit:]]+` which implies one or more occurrences of decimal digits, the same as `[0-9]+`. So, the given regular expression finds a match only if the line of text contains only decimal digits and nothing more. Due to this reason, lines of text like *AA111*, *22.22*, *a1234z*, etc. are not selected.

```
# g++ regex7.cc
# ./a.out
111222
123456
111
2222
33333
#
```

Figure 4: Regular expressions for numbers

Now it is time for us to wind up the discussion. Like the previous two articles in this series, I have covered the use of regular expressions in a particular programming language as well as some other aspects of the programming language that will affect the usage of regular expressions. In this article, the lengthy discussion about the standards of C++ was absolutely essential, without which you might blindly apply regular expressions on all standards of C++ without considering the subtle differences between them. The topics on regular expressions discussed in this article may not be comprehensive but they provide an adequate basis for any good C++ programmer to build up from. In the next part of this series we will discuss the use of regular expressions in yet another programming language, maybe one that is much used on the Internet and the World Wide Web. 

By: Deepu Benson

The author has nearly 16 years of programming experience. He is a free software enthusiast and his area of interest is theoretical computer science. The author maintains a technical blog at www.computingforbeginners.blogspot.in and can be reached at deepumb@hotmail.com.